ZIP自解压

介绍

这个是文件上传业务的其中之一，实际上是归在文件上传里面的，但是因为一些特性把这个单独拿出来讲，简单介绍下ZIP自解压，字面意思就算压缩包解压操作，部分系统因为业务考虑允许上传压缩包，有点会默认解压，有的要手动解压，那么只要我们能控制上传的压缩包和解压就有可能出现任意文件上传

简单的漏洞demo

工具类

```
public class ZipUtils {

    /**
     * 解压ZIP文件到指定目录
     * @param zipFile ZIP文件路径
     * @param destDir 解压目标目录
     * @throws IOException
     */
    public static void unzip(String zipFile, String destDir) throws IOException {
        File dir = new File(destDir);
        if (!dir.exists()) {
            dir.mkdirs();
        }

        try (ZipInputStream zipIn = new ZipInputStream(new
FileInputStream(zipFile))) {
            ZipEntry entry = zipIn.getNextEntry();

            while (entry != null) {
                String filePath = destDir + File.separator + entry.getName();
                if (!entry.isDirectory()) {
                    extractFile(zipIn, filePath);
                } else {
                    File subDir = new File(filePath);
                    subDir.mkdirs();
                }
                zipIn.closeEntry();
                entry = zipIn.getNextEntry();
            }
        }
    }

    private static void extractFile(ZipInputStream zipIn, String filePath) throws
IOException {
        try (BufferedOutputStream bos = new BufferedOutputStream(new
FileOutputStream(filePath))) {
            byte[] bytesIn = new byte[4096];
            int read = 0;
            while ((read = zipIn.read(bytesIn)) != -1) {
                bos.write(bytesIn, 0, read);
            }
```

```
        }
    }
}
```

实现类

```
public class ZipUtils {

    /**
     * 解压ZIP文件到指定目录
     * @param zipFile ZIP文件路径
     * @param destDir 解压目标目录
     * @throws IOException
     */
    public static void unzip(String zipFile, String destDir) throws IOException {
        // 检查目标目录是否存在，如果不存在则创建
        File dir = new File(destDir);
        if (!dir.exists()) {
            dir.mkdirs();
        }

        // 使用try-with-resources语句自动管理ZipInputStream的关闭
        try (ZipInputStream zipIn = new ZipInputStream(new
FileInputStream(zipFile))) {
            ZipEntry entry = zipIn.getNextEntry();

            // 遍历ZIP文件中的每个条目
            while (entry != null) {
                String filePath = destDir + File.separator + entry.getName();
                // 判断当前条目是否为目录，如果不是则解压文件，否则创建目录
                if (!entry.isDirectory()) {
                    extractFile(zipIn, filePath);
                } else {
                    File subDir = new File(filePath);
                    subDir.mkdirs();
                }
                // 关闭当前条目并移动到下一个条目
                zipIn.closeEntry();
                entry = zipIn.getNextEntry();
            }
        }
    }

    /**
     * 从ZipInputStream中提取文件并写入指定路径
     *
     * @param zipIn ZIP输入流
     * @param filePath 要写入的文件路径
     * @throws IOException 如果写入文件时发生I/O错误
     */
    private static void extractFile(ZipInputStream zipIn, String filePath) throws
IOException {
        try (BufferedOutputStream bos = new BufferedOutputStream(new
FileOutputStream(filePath))) {
            byte[] bytesIn = new byte[4096];
```

```
            int read = 0;
            while ((read = zipIn.read(bytesIn)) != -1) {
                bos.write(bytesIn, 0, read);
            }
        }
    }
}
```

代码很简单，就是上传zip，然后通过自定义的unzip方法遍历压缩包中的文件，然后通过extractFile方法读取文件流写入到新文件中，那么这里肯定是有问题的，因为没有对我们上的zip包中的文件名和后缀进行校验，如果我们上传的zip中存在个../../caigo.jsp文件，那不是会照成跨目录的恶意文件上传吗?

实际业务结合

但我们简单了解了漏洞成因，我们来思考会出现解压功能的地方，其实也就是我们的黑盒测试点了

1、文件管理系统/模块，在文件管理系统或者是网站文件管理模块中基本都允许上传压缩包，并且有一定概率允许执行解压，这个就是看源码了

2、网站的更新功能，部分源码支持在线安装更新包

3、网站插件安装，部分网站需要额外拓展功能的情况下，厂商可能会推出插件安装功能，供用户增加功能

那么简单来说就算存在压缩包解压的功能点都有可能存在这个漏洞

安全业务实现

我们接下来以开发的角度来思考一些东西

关于文件解压的预防策略，通过前面的演示我们知道产生危害的点主要是文件名存在..可能导致目录穿越，文件存在恶意后缀，可能导致恶意代码解析问题，那么比如说我开发的是个文件管理系统，我就可以限制文件名出现..，和禁用一些恶意文件后缀，也就是黑名单

```
    private static boolean isValidZipEntryName(String name) {
        return !name.contains("..") && !name.contains(File.separator + "..");
    }

    /**
     * 校验是否不是 JSP/JSX 文件
     */
    private static boolean isNotWebScriptFile(String name) {
        return !(name.endsWith(".jsp") || name.endsWith(".jspx"));
    }
```

那么就能预防这个问题

那么如果是插件功能点或者是更新包，那么多数插件和更新包本身就是可执行程序，例如jar包或者jsp文件，那么在这种情况下黑名单校验文件会和我们的业务需要冲突，那么有什么好的办法呢

1.通过校验发行插件或更新包的md5值，判断文件是否被篡改过，判断是否安装

2.数字签名校验方案

还有其它预防文件篡改的思路，这里就不细说了

实战案例

publiccms

MCMS

```
@ApiOperation(value = "处理post请求上传模板文件")

    @PostMapping("/uploadTemplate")

    @ResponseBody

    public ResultData uploadTemplate(Bean bean, boolean uploadFloderPath,
HttpServletResponse res) throws IOException {

        String uploadTemplatePath = MSProperties.upload.template;

        //非法路径过滤

        if(checkUploadPath(bean)){

            return ResultData.build().error(getResString("err.error", new
String[]{getResString("file.type")}));

        }

        if (StringUtils.isEmpty(bean.getUploadPath())) {

            bean.setUploadPath(uploadTemplatePath + File.separator +
BasicUtil.getApp().getAppId());

        }



        Config config = new Config(bean.getUploadPath(), bean.getFile(),null,
uploadFloderPath, bean.isRename());

        return this.uploadTemplate(config);

    }



    protected boolean checkUploadPath(Bean bean){

        return (bean.getUploadPath()!=null&&
(bean.getUploadPath().contains("../")||bean.getUploadPath().contains("..\\")));

    }
```

任意文件写入

介绍

任意文件写入(Arbitrary File Write)是一种常见的安全漏洞，攻击者利用该漏洞可以向服务器文件系统的任意位置写入恶意内容，可能导致严重后果

任意文件写入漏洞通常发生在以下场景：

1. 应用程序未对用户提供的文件名或路径进行充分验证
2. 使用用户可控数据拼接文件路径
3. 未限制文件写入的目录位置
4. 未校验文件内容的安全性

实际业务结合

文件编写类的，模板文件修改，静态页面修改等等

白盒关键词

核心文件写入类和方法

- `FileOutputStream` - 文件输出流类
- `FileWriter` - 文件写入类
- `BufferedWriter` - 缓冲写入类
- `PrintWriter` - 格式化输出类
- `Files.write()` - Java NIO的便捷写入方法
- `Files.writeString()` - Java 11+的字符串写入方法

关键写入方法

- `write()` - 基本写入方法
- `append()` - 追加内容方法
- `flush()` - 刷新缓冲区
- `close()` - 关闭流

文件路径处理

- `File` - 文件操作类
- `Path` - NIO路径接口
- `Paths.get()` - 获取Path对象

安全业务实现

和文件上传一样

实战案例

MCMS

DocSys